

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 6**



GRAPH AND TREE

Oleh:

Muhammad Hafi

NIM. 2510817310011

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
Mei 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 6

Laporan Praktikum Algoritma & Struktur Data Modul 6: Graph and Treea ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikun ini dikerjakan oleh:

Nama Praktikan : Muhammad Hafi
NIM : 25108173100111

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
Soal 1: Konversi Adjacency List ke AdjacencyMatrix.....	6
Deskripsi Soal.....	6
A. Source Code.....	8
B. Output Program.....	9
C. Pembahasan.....	9
D. Analisis	10
Soal 2: Konversi Adjacency Matrix ke Adjacency List	12
A. Source Code.....	14
B. Ouput Program.....	15
C.Pembahasan.....	15
Soal 3: BFS: Jarak Terpendek Keluar Labirin.....	17
A. Source Code.....	19
B. Ouput Program.....	20
C. Pembahasan.....	20
D. Analisis	22
Soal 4: DFS: Menghitung Semua Jalur Keluar Labirin	23
A. Source Code.....	25
B. Ouput Program.....	26
C. Pembahasan.....	26
D. Analisis	29
Soal 5: Tree Traversal.....	30
A. Source Code.....	32
B. Output Program.....	34
C. Pembahasan.....	34
D. Analisis	37

DAFTAR GAMBAR

Gambar 1. Output Soal1.cpp	9
Gambar 2. Ouput Soal2.cpp	15
Gambar 3. Ouput Soal3.cpp	20
Gambar 4. Ouput Soal4.cpp	26
Gambar 5. Ouput Soal5.cpp	34

DAFTAR TABEL

Table 1. Source Code Soal1.cpp.....	8
Table 2. Source Code Soal2.cpp.....	14
Table 3. Source Code Soal3.cpp	19
Table 4. Source Code Soal4.cpp.....	25
Table 5. Source Code Soal5.cpp.....	32

Soal 1: Konversi Adjacency List ke AdjacencyMatrix

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah directed weighted graph yang terdiri dari N vertex dan M edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge.

Setiap edge ditulis sebagai U, V, W , yang berarti terdapat edge berarah dari vertex U menuju vertex V dengan bobot W . Karena graph bersifat berarah, edge U, V, W tidak otomatis berarti terdapat edge dari V menuju U .

Tugas Anda adalah mengubah representasi graph tersebut menjadi adjacency matrix. Jika terdapat edge dari vertex ke- i menuju vertex ke- j , maka nilai matrix pada baris i dan kolom j adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

Format Input

Input diberikan dalam format berikut.

```
N
V1 V2 ... VN
M
U1 V1 W1
U2 V2 W2
...
UM VM WM
```

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- M adalah jumlah edge.
- U_i dan V_i adalah vertex asal dan vertex tujuan pada edge ke- i .
- W_i adalah bobot edge ke- i .

Format Output

Cetak adjacency matrix dengan format berikut.

```
Adjacency Matrix:
V1 V2 ... VN
V1 a11 a12 ... a1N
V2 a21 a22 ... a2N
...
VN aN1 aN2 ... aNN
```

Nilai a_{ij} adalah bobot edge dari vertex V_i menuju vertex V_j . Jika edge tersebut tidak ada, nilai a_{ij} adalah 0.

Batasan

- $2 \leq N \leq 10$
- $0 \leq M \leq N * (N - 1)$
- $1 \leq W_i \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Graph tidak memiliki **self-loop**.
- Graph tidak memiliki edge duplikat dengan arah yang sama.
- Graph bersifat berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 6 A B 4 A C 2 B D 7 C B 1 C E 6 D C 3	Adjacency Matrix: A B C D E A 0 4 2 0 0 B 0 0 0 7 0 C 0 1 0 0 6 D 0 0 3 0 0 E 0 0 0 0 0

Penjelasan Contoh

Edge A B 4 menghasilkan nilai 4 pada baris A kolom B. Karena graph berarah, nilai pada baris B kolom A tetap 0 selama tidak ada edge dari B menuju A.

Edge D C 3 menghasilkan nilai 3 pada baris D kolom C. Vertex E tidak memiliki edge keluar, sehingga seluruh nilai pada baris E adalah 0.

A. Source Code

Table 1. Source Code Soal1.cpp

1.	#include <iostream>
2.	#include <vector>
3.	
4.	int main() {
5.	int N, M;
6.	
7.	if (!(std::cin >> N)) return 0;
8.	std::vector<char> labels(N);
9.	for (int i = 0; i < N; ++i) {
10.	std::cin >> labels[i];
11.	}
12.	std::vector<std::vector<int>>> matrix(N, std::vector<int>(N,
13.	0));
14.	
15.	if (!(std::cin >> M)) return 0;
16.	for (int i = 0; i < M; ++i) {
17.	char u, v;
18.	int w;
19.	if (std::cin >> u >> v >> w) {
20.	int u_idx = -1, v_idx = -1;
21.	for (int k = 0; k < N; ++k) {
22.	if (labels[k] == u) u_idx = k;
23.	if (labels[k] == v) v_idx = k;
24.	}
25.	if (u_idx != -1 && v_idx != -1) {
26.	matrix[u_idx][v_idx] = w;
27.	}
28.	}
29.	}
30.	
31.	std::cout << "Adjacency Matrix:\n";
32.	for (int i = 0; i < N; ++i) std::cout << " " << labels[i];
33.	std::cout << std::endl;
34.	
35.	for (int i = 0; i < N; ++i) {
36.	std::cout << labels[i];
37.	for (int j = 0; j < N; ++j) {
38.	std::cout << " " << matrix[i][j];
39.	}
40.	std::cout << std::endl;
41.	}
42.	return 0;

43.

}

B. Output Program

```
PS C:\Users\LOQ\OneDrive\Documents\GitHub\module-6-graph-and-tree-hafimuhammad-main> cd "c:\Users\LOQ\OneDrive\Documents\GitHub\module-6-graph-and-tree-hafimuhammad-main\" ; if ($?) { g++ Soal1.cpp -o Soal1 } ; if ($?) { .\Soal1 }
5
A B C D E 6
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0
```

Gambar 1. Output Soal1.cpp

C. Pembahasan

`#include <iostream>` dan `#include <vector>` ini dipakai buat masukin pustaka standar C++ supaya program bisa melakukan input-output dan menggunakan struktur data vector yang fleksibel.

`int main()` { merupakan titik awal program ini mulai dijalankan.

`int N, M;` di sini kita deklarasikan dua variabel integer untuk menyimpan jumlah vertex (N) dan jumlah sisi (M).

`if (!(std::cin >> N)) return 0;` baris ini membaca nilai N dari user, dan kalau inputnya gagal/kosong, program langsung berhenti.

`std::vector<char> labels(N);` membuat vector berukuran N untuk menampung karakter label dari masing-masing vertex.

`for (int i = 0; i < N; ++i) { std::cin >> labels[i]; }` ini perulangan buat ngisi setiap posisi di vector labels dengan karakter yang diinput user.

`std::vector<std::vector<int>> matrix(N, std::vector<int>(N, 0));` membuat matriks dua dimensi ukuran N x N, semua isinya langsung diinisialisasi 0 dulu.

`if (!(std::cin >> M)) return 0;` membaca jumlah sisi yang mau diproses, sekaligus ada pengecekan kalau inputnya kosong.

`for (int i = 0; i < M; ++i) {` mulai perulangan sebanyak M kali, masing-masing buat proses satu sisi graf.

`char u, v; int w;` deklarasi variabel buat nampung label vertex asal (u), vertex tujuan (v), dan bobot sisinya (w).

`if (std::cin >> u >> v >> w) {` baris ini baca ketiga nilai itu dari input; isi blok di dalamnya cuma jalan kalau pembacaannya berhasil.

`int u_idx = -1, v_idx = -1;` siapin variabel indeks buat nyari posisi label u dan v di dalam matriks.

`for (int k = 0; k < N; ++k) {` di sini program nyari posisi indeks dari label yang diinput, caranya dengan ngecek satu-satu isi vector `labels`.

`if (labels[k] == u) u_idx = k;` kalau label u ketemu, posisinya disimpan ke `u_idx`.

`if (labels[k] == v) v_idx = k;` sama, kalau label v ketemu, posisinya disimpan ke `v_idx`.

`if (u_idx != -1 && v_idx != -1) { matrix[u_idx][v_idx] = w; }` kalau dua label itu valid (artinya ketemu, ditandai bukan -1), barulah bobot `w` dimasukkan ke sel matriks yang sesuai. Perlu diperhatikan, baris ini posisinya **di luar** loop pencarian `for (k...)`, jadi pengecekan dan pengisian matriksnya baru dilakukan setelah semua label selesai dibandingkan, bukan di tengah-tengah proses pencarian.

Setelah itu kurung kurawal penutup menutup blok `if (std::cin >> u >> v >> w)`, lalu disusul kurung penutup buat blok `for (i...)`-nya, jadi satu sisi sudah selesai diproses sebelum lanjut ke sisi berikutnya.

`std::cout << "Adjacency Matrix:\n";` nyetak judul ke layar sebagai info hasil programnya.

`for (int i = 0; i < N; ++i) std::cout << " " << labels[i]; std::cout << std::endl;` ini nyetak baris label kolom dulu, biar matriksnya nanti lebih enak dibaca.

`for (int i = 0; i < N; ++i) {` mulai perulangan buat nyetak isi matriks baris demi baris.

`std::cout << labels[i];` nyetak label vertex di awal tiap baris matriks.

`for (int j = 0; j < N; ++j) { std::cout << " " << matrix[i][j]; }` nyetak nilai bobot yang ada di setiap kolom pada baris ke-`i` itu.

`std::cout << std::endl;` } pindah baris baru setelah satu baris matriks selesai dicetak, lalu lanjut ke baris vertex berikutnya.

`return 0; }` menandakan fungsi `main` selesai dan programnya berhasil dijalankan sampai akhir.

D. Analisis

Algoritma ini membangun adjacency matrix melalui tiga tahap utama: pemetaan, translasi, dan konstruksi. Pertama, program menyimpan label vertex ke dalam vektor sebagai referensi indeks. Kedua, saat membaca sisi, program melakukan pencarian linear untuk menerjemahkan label ke koordinat matriks (baris, kolom). Ketiga, bobot sisi ditempatkan pada koordinat tersebut dalam matriks $N \times N$ yang telah diinisialisasi dengan nol. Hasil akhirnya adalah tabel numerik yang

merepresentasikan struktur graf secara terarah. Kompleksitas waktu dari algoritma ini adalah $O(M \times N)$, di mana M adalah jumlah sisi dan N adalah jumlah vertex. Hal ini disebabkan oleh proses pencarian label, untuk setiap M sisi yang dibaca, program melakukan perulangan for hingga N kali guna menemukan posisi indeks dari dua vertex yang terlibat. adanya deklarasi matriks `matrix(N, std::vector(N, 0))` menyebabkan matriks ini menjadi variabel dominan yang menentukan batas atas penggunaan ruang memori. Sehingga penggunaan memori sekaligus kompleksitas ruangnya sebesar $O(N^2)$.

Soal 2: Konversi Adjacency Matrix ke Adjacency List

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah directed weighted graph yang direpresentasikan dalam bentuk adjacency matrix berukuran $N \times N$. Setiap vertex memiliki label berupa satu karakter unik.

Nilai matrix pada baris i dan kolom j menunjukkan bobot edge dari vertex ke- i menuju vertex ke- j . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- i menuju vertex ke- j .

Tugas Anda adalah mengubah adjacency matrix tersebut menjadi adjacency list. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edge-nya. Urutan vertex tujuan mengikuti urutan label pada input.

Format Input

Input diberikan dalam format berikut.

```
N
V1 V2 ... VN A11 A12 ... A1N A21 A22 ... A2N
...
AN1 AN2 ... ANN
```

Keterangan:

N adalah jumlah vertex.

V_i adalah label vertex ke- i .

A_{ij} adalah nilai matrix pada baris ke- i dan kolom ke- j .

Jika $A_{ij} > 0$, maka terdapat edge dari V_i menuju V_j dengan bobot A_{ij} .

Jika $A_{ij} = 0$, maka tidak terdapat edge dari V_i menuju V_j .

Format Output

Cetak adjacency list untuk setiap vertex dengan format berikut.

```
Adjacency List:
V1 -> (X1,w1), (X2,w2), ...
V2 -> (X1,w1), (X2,w2), ...
...
VN -> ...
```

Jika sebuah vertex tidak memiliki edge keluar, cetak tanda - setelah simbol ->.

Constraints

$$2 \leq N \leq 10$$

$$0 \leq A_{ij} \leq 1000$$

Label vertex berupa karakter alfabet kapital A sampai Z.

Semua label vertex dijamin unik.

Nilai diagonal utama matrix dijamin 0.

Nilai 0 berarti tidak ada edge.

Nilai positif berarti bobot edge.

Matrix merepresentasikan graph berarah dan berbobot.

Contoh Kasus

Input	Output
5	Adjacency List:
A B C D E	A -> (B,4), (C,2)
0 4 2 0 0	B -> (D,7)
0 0 0 7 0	C -> (B,1), (E,6)
0 1 0 0 6	D -> (C,3)
0 0 3 0 0	E -> -
0 0 0 0 0	

Penjelasan Contoh

Pada baris vertex A , nilai positif muncul pada kolom B dan C . Karena itu, adjacency list untuk

A adalah $(B, 4), (C, 2)$.

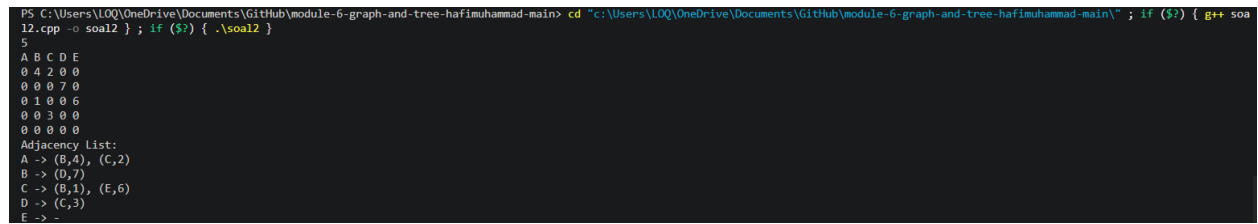
Pada baris vertex E , semua nilai bernilai 0, sehingga vertex E tidak memiliki edge keluar dan output-nya adalah $E \rightarrow -$.

A. Source Code

Table 2. Source Code Soal2.cpp

1.	#include <iostream>
2.	#include <vector>
3.	
4.	int main() {
5.	int N;
6.	if (!(std::cin >> N) N <= 0) return 0;
7.	
8.	std::vector<char> labels(N);
9.	for (int i = 0; i < N; ++i) {
10.	std::cin >> labels[i];
11.	}
12.	
13.	std::vector<std::vector<int>> matrix(N,
	std::vector<int>(N));
14.	for (int i = 0; i < N; ++i) {
15.	for (int j = 0; j < N; ++j) {
16.	std::cin >> matrix[i][j];
17.	}
18.	}
19.	
20.	std::cout << "Adjacency List:\n";
21.	for (int i = 0; i < N; ++i) {
22.	std::cout << labels[i] << " -> ";
23.	bool first = true;
24.	bool Edge = false;
25.	for (int j = 0; j < N; ++j) {
26.	if (matrix[i][j] != 0) {
27.	if (!first) std::cout << ", ";
28.	std::cout << "(" << labels[j] << ", " <<
	matrix[i][j] << ")";
29.	first = false;
30.	Edge = true;
31.	}
32.	}
33.	if (!Edge) {
34.	std::cout << "-";
35.	}
36.	std::cout << std::endl;
37.	}
38.	
39.	return 0;
40.	}

B. Ouput Program



```
PS C:\Users\LOQ\OneDrive\Documents\Github\module-6-graph-and-tree-hafimuhammad-main> cd "c:\Users\LOQ\OneDrive\Documents\Github\module-6-graph-and-tree-hafimuhammad-main\" ; if ($?) { g++ soa
12.cpp -o soal2 } ; if ($?) { .\soal2 }
5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency List:
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -
```

Gambar 2. Ouput Soal2.cpp

C. Pembahasan

`#include <iostream>` dan `#include <vector>` ini dipakai untuk membuat pustaka standar C++ supaya program bisa melakukan input dan output, untuk menggunakan struktur data vector yang fleksibel.

`int main()` { titik awal program ini mulai dijalankan.

`int N;` deklarasi variabel integer buat nyimpan jumlah vertex.

`if (!(std::cin >> N) || N <= 0) return 0;` baris ini baca nilai N dari user, dan langsung berhenti kalau inputnya gagal **atau** kalau N-nya nol/negatif jadi ada validasi tambahan dibanding kode sebelumnya, supaya nggak ada kasus N negatif yang bikin program error pas alokasi vector.

`std::vector<char> labels(N);` bikin vector berukuran N buat nampung karakter label tiap vertex.

`for (int i = 0; i < N; ++i) { std::cin >> labels[i]; }` perulangan buat ngisi tiap posisi di vector labels dengan karakter yang diinput user.

`std::vector<std::vector<int>> matrix(N, std::vector<int>(N));` bikin matriks dua dimensi ukuran N x N. Beda dengan kode sebelumnya, di sini matriksnya nggak diinisialisasi 0 secara eksplisit, soalnya isinya bakal langsung diisi penuh dari input user di baris selanjutnya jadi nggak perlu nilai default.

`for (int i = 0; i < N; ++i) { for (int j = 0; j < N; ++j) { std::cin >> matrix[i][j]; } }` ini dua loop bersarang yang baca seluruh isi matriks dari input user, baris demi baris, kolom demi kolom — jadi user di sini langsung kasih adjacency matrix yang utuh, bukan dalam bentuk list sisi (u, v, w) kayak soal sebelumnya.

`std::cout << "Adjacency List:\n";` nyetak judul ke layar sebagai info hasil programnya.

`for (int i = 0; i < N; ++i) {` mulai perulangan buat mroses tiap vertex satu per satu, baris ini bakal nentuin vertex mana yang jadi titik asal.

`std::cout << labels[i] << " -> ";` nyetak label vertex yang sedang diproses, diikuti tanda panah buat nunjukin awal dari daftar tujuannya.

`bool first = true; bool Edge = false;` dua variabel boolean disiapkan di sini — `first` buat ngatur kapan koma pemisah perlu dicetak, dan `Edge` buat nandain apakah vertex ini punya minimal satu sisi keluar atau nggak.

`for (int j = 0; j < N; ++j) {` loop dalam buat ngecek satu-satu kolom di baris ke-`i`, nyari semua vertex tujuan yang terhubung dari vertex `i`.

`if (matrix[i][j] != 0) {` kalau nilai di sel itu bukan nol, artinya ada sisi dari vertex `i` ke vertex `j` dengan bobot sebesar nilai itu.

`if (!first) std::cout << ", ";` kalau ini bukan sisi pertama yang ditemukan, cetak koma dulu sebagai pemisah sebelum nyetak sisi berikutnya.

`std::cout << "(" << labels[j] << ", " << matrix[i][j] << ")";` nyetak pasangan vertex tujuan dan bobotnya dalam format (`label`, `bobot`).

`first = false; Edge = true;` update status, nandain bahwa udah ada minimal satu sisi yang ketemu dan dicetak.

`if (!Edge) { std::cout << "-"; }` setelah loop dalam selesai, kalau ternyata nggak ada satu pun sisi yang ketemu buat vertex `i` (`Edge` masih `false`), cetak tanda "-" sebagai penanda vertex itu nggak punya tujuan keluar.

`std::cout << std::endl; }` pindah baris baru setelah satu vertex selesai diproses, lalu lanjut ke vertex berikutnya.

`return 0; }` fungsi main selesai.

D. Analisis

Algoritma ini menggunakan metode Breadth First (BFS) buat mencari jalur terpendek dalam grid dua dimensi, dengan bekerja secara sistematis menyebar dari titik awal ke arah adjacent vertices terdekat secara berlapis. Proses diawali dengan memasukkan titik awal ke dalam antrian, lalu secara berulang mengambil vertex dari antrian tersebut untuk diperiksa adjacent verticesnya ke empat arah mata angin. Algoritma memanfaatkan matriks `visited` untuk mencatat sel yang sudah pernah dilewati, sehingga mencegah pengulangan proses pada sel yang sama dan menghindari looping yang tidak perlu. Karena sifat BFS yang menjelajahi setiap sel berdasarkan jarak langkah dari titik awal, vertex tujuan yang ditemukan pertama kali dipastikan merupakan jalur terpendek yang mungkin, dengan total kompleksitas waktu sebesar $O(R \times C)$ karena setiap sel pada grid akan dikunjungi maksimal satu kali. semua struktur data utama yang digunakan skalanya berbanding lurus dengan jumlah sel dalam grid ($N = R \times C$), maka total kompleksitas ruangnya adalah $O(R \times C)$.

Soal 3: BFS: Jarak Terpendek Keluar Labirin

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Sebuah labirin direpresentasikan sebagai grid berukuran $R \times C$. Setiap sel pada grid memiliki nilai sebagai berikut.

0 menyatakan jalan yang dapat dilewati.

1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma **Breadth-First Search** (BFS).

Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

Format Input

Input diberikan dalam format berikut.

```
R C
G11 G12 ... G1C G21 G22 ... G2C
...
GR1 GR2 ... GRC SR SC
FR FC
```

Keterangan:

R adalah jumlah baris grid.

C adalah jumlah kolom grid.

G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .

(S_R, S_C) adalah posisi awal.

(F_R, F_C) adalah posisi tujuan. Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

X

Keterangan:

X adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.

Jika posisi tujuan tidak dapat dicapai, cetak -1.

Constraints

$$1 \leq R \leq 100$$

$$1 \leq C \leq 100$$

G_{ij} hanya bernilai 0 atau 1.

$$0 \leq S_R < R$$

$$0 \leq S_C < C$$

$$0 \leq F_R < R$$

$$0 \leq F_C < C$$

Sel start dan finish dijamin bernilai 0.

Pergerakan hanya boleh dilakukan ke sel bernilai 0.

Algoritma yang digunakan wajib BFS.

Contoh Kasus

Input	Output
8 10 0 0 0 0 1 0 0 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0 1 0 0 0 0 0 1 0 0 1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 1 1 0 0 0 0 0 1 0 0 0 1 0 0 1 1 0 0 0 1 0 0 0 0 0 7 9	16

Penjelasan Contoh

Posisi awal berada pada $(0, 0)$ dan posisi tujuan berada pada $(7, 9)$. Labirin memiliki beberapa percabangan dan jalan buntu, misalnya cabang menuju area kiri bawah dan cabang menuju sel $(2, 1)$.

Dengan BFS, setiap sel diproses berdasarkan jarak terdekat dari start. Jarak minimum dari

$(0, 0)$ ke $(7, 9)$ adalah 16 langkah, sehingga output yang dicetak hanya 16.

A. Source Code

Table 3. Source Code Soal3.cpp

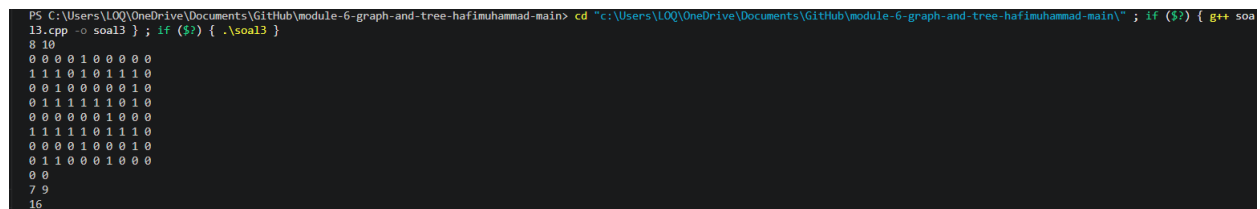
1.	#include <iostream>
2.	#include <vector>
3.	
4.	using namespace std;
5.	struct Node {
6.	int r, c, dist;
7.	};
8.	
9.	int main() {
10.	int R, C;
11.	if (!(cin >> R >> C) R <= 0 C <= 0) return 0;
12.	
13.	vector<vector<int>> grid(R, vector<int>(C));
14.	for (int i = 0; i < R; ++i) {
15.	for (int j = 0; j < C; ++j) {
16.	cin >> grid[i][j];
17.	}
18.	}
19.	
20.	int SR, SC, FR, FC;
21.	if (!(cin >> SR >> SC >> FR >> FC)) return 0;
22.	if (SR < 0 SR >= R SC < 0 SC >= C FR < 0
23.	FR >= R FC < 0 FC >= C) {
24.	return 0;
25.	}
26.	vector<Node> q;
27.	int head = 0;
28.	
29.	vector<vector<bool>> visited(R, vector<bool>(C, false));
30.	
31.	q.push_back({SR, SC, 0});
32.	visited[SR][SC] = true;
33.	
34.	int dr[] = {-1, 1, 0, 0};
35.	int dc[] = {0, 0, -1, 1};
36.	
37.	while (head < q.size()) {
38.	Node curr = q[head];
39.	head++;
40.	
41.	if (curr.r == FR && curr.c == FC) {
42.	cout << curr.dist << endl;
43.	return 0;
44.	}

```

45.
46. for (int i = 0; i < 4; ++i) {
47.     int nr = curr.r + dr[i];
48.     int nc = curr.c + dc[i];
49.
50.     if (nr >= 0 && nr < R && nc >= 0 && nc < C
51.         && grid[nr][nc] == 0 && !visited[nr][nc]) {
52.         visited[nr][nc] = true;
53.         q.push_back({nr, nc, curr.dist + 1});
54.     }
55. }
56. }
57. cout << -1 << endl;
58. return 0;
59. }

```

B. Ouput Program



```

PS C:\Users\LOQ\OneDrive\Documents\GitHub\module-6-graph-and-tree-hafimuhammad-main> cd "c:\Users\LOQ\OneDrive\Documents\GitHub\module-6-graph-and-tree-hafimuhammad-main\" ; if ($?) { g++ soa
l3.cpp -o soal3 } ; if ($?) { .\soal3 }
8 10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 0 1 0
0 0 0 0 0 1 0 0 0
1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 1 0
0 1 1 0 0 0 1 0 0 0
0 0
7 9
16

```

Gambar 3. Ouput Soal3.cpp

C. Pembahasan

`#include <iostream>` dan `#include <vector>` ini dipakai untuk membuat pustaka standar C++ supaya program bisa melakukan input output dan menggunakan struktur data vector yang fleksibel.

`using namespace std;` ini bikin kita kda perlu nulis `std::` di depan setiap fungsi/objek standar library.

`struct Node { int r, c, dist; };` di sini didefinisikan `struct Node` yang isinya tiga atribut: `r` dan `c` buat nyimpen posisi baris dan kolom suatu sel di grid, sama `dist` buat nyimpen jarak (jumlah langkah) dari titik awal sampai ke sel itu. Struct ini nantinya dipakai sebagai elemen di antrean BFS.

`int main() {` titik awal program mulai dijalankan.

`int R, C;` deklarasi variabel buat nyimpen jumlah baris (R) dan kolom (C) dari grid.

`if (!(cin >> R >> C) || R <= 0 || C <= 0) return 0;` baca nilai R dan C, lalu langsung berhenti kalau pembacaannya gagal atau kalau salah satu nilainya nol/negatif soalnya ukuran grid yang nggak valid bisa bikin program error pas alokasi memori.

`vector<vector<int>> grid(R, vector<int>(C));` bikin grid dua dimensi ukuran $R \times C$ buat nyimpen data petanya (misalnya 0 buat jalan yang bisa dilewati, dan nilai lain buat penghalang).

`for (int i = 0; i < R; ++i) { for (int j = 0; j < C; ++j) { cin >> grid[i][j]; } }` dua loop bersarang buat baca isi grid dari input user, baris demi baris, kolom demi kolom.

`int SR, SC, FR, FC;` deklarasi variabel buat nyimpen koordinat titik awal (SR, SC) dan titik tujuan (FR, FC).

`if (!(cin >> SR >> SC >> FR >> FC)) return 0;` baca keempat koordinat itu, langsung berhenti kalau pembacaannya gagal.

`if (SR < 0 || SR >= R || SC < 0 || SC >= C || FR < 0 || FR >= R || FC < 0 || FC >= C) { return 0; }` ini validasi tambahan, mastiin koordinat titik awal dan tujuan yang diinput user masih dalam batas ukuran grid (nggak boleh negatif atau lebih besar dari R/C) — kalau salah satu koordinatnya di luar batas, program berhenti.

`vector<Node> q;` bikin vector `q` yang berfungsi sebagai antrean (queue) buat nyimpen sel-sel yang bakal diperiksa selama proses BFS.

`int head = 0;` variabel `head` ini dipakai sebagai penanda posisi "depan" antrean jadi alih-alih pakai `std::queue` beneran (yang ngehapus elemen depan tiap kali di-pop), di sini antreannya disimulasikan pakai vector biasa, dan `head` nunjukin elemen mana yang lagi diproses sekarang.

`vector<vector<bool>> visited(R, vector<bool>(C, false));` bikin matriks boolean ukuran $R \times C$, semua diinisialisasi `false`, buat nyatet sel mana aja yang udah pernah dikunjungi selama BFS.

`q.push_back({SR, SC, 0});` titik awal dimasukin ke antrean sebagai elemen pertama, dengan jarak awal 0.

`visited[SR][SC] = true;` titik awal langsung ditandai sebagai udah dikunjungi, supaya nggak diproses ulang nanti.

`int dr[] = {-1, 1, 0, 0}; int dc[] = {0, 0, -1, 1};` dua array ini nyimpen pola pergeseran koordinat buat ngecek empat arah tetangga (atas, bawah, kiri, kanan) dari suatu sel — pasangan `dr[i]` dan `dc[i]` di indeks yang sama ngewakilin satu arah gerakan.

`while (head < q.size())` { loop utama BFS ini berjalan terus menerus selama masih ada elemen di antrean yang belum diproses (ditandai dengan `head` belum nyampe ukuran `q`).

`Node curr = q[head]; head++;` ambil elemen di posisi depan antrean (sesuai `head`), simpan ke variabel `curr`, lalu majukan `head` supaya elemen ini dianggap udah diproses dan nggak diulang.

`if (curr.r == FR && curr.c == FC) { cout << curr.dist << endl; return 0; }` kalau sel yang sedang diproses ternyata sama dengan titik tujuan, langsung cetak jaraknya dan program berhenti — soalnya BFS jamin kalau titik tujuan ditemukan, jaraknya itu udah pasti yang paling pendek.

`for (int i = 0; i < 4; ++i) {` perulangan buat ngecek keempat arah tetangga dari sel yang sedang diproses.

`int nr = curr.r + dr[i]; int nc = curr.c + dc[i];` ngitung koordinat baris dan kolom dari tetangga ke arah ke-*i*, berdasarkan posisi sekarang ditambah pola pergeseran yang udah disiapin di array `dr` dan `dc`.

`if (nr >= 0 && nr < R && nc >= 0 && nc < C && grid[nr][nc] == 0 && !visited[nr][nc])` { ini pengecekan empat syarat sekaligus sebelum tetangga itu boleh diproses: koordinatnya harus masih dalam batas grid (nggak keluar baris/kolom), selnya harus berupa jalan yang bisa dilewati (nilai grid-nya 0), dan selnya belum pernah dikunjungi sebelumnya.

`visited[nr][nc] = true;` kalau semua syarat terpenuhi, sel tetangga itu langsung ditandai udah dikunjungi.

`q.push_back({nr, nc, curr.dist + 1});` sel tetangga itu dimasukin ke antrian buat diproses nanti, dengan jaraknya ditambah satu dari jarak sel sekarang.

`}` } dua kurung kurawal ini nutup blok `if` pengecekan syarat tetangga, lalu nutup loop `for` pengecekan empat arah.

`}` kurung ini nutup loop `while` utama, menandakan satu iterasi pemrosesan satu sel BFS udah selesai.

`cout << -1 << endl;` baris ini cuma bakal dieksekusi kalau loop `while` udah selesai total (antrian kosong) tapi titik tujuan nggak pernah ketemu — jadi -1 dicetak sebagai tanda kalau jalur ke tujuan nggak ada.

`return 0; }` fungsi main selesai.

D. Analisis

algoritma yang digunakan adalah DFS dengan backtracking yang berfungsi menghitung seluruh jumlah rute dari titik awal ke tujuan. Cara kerjanya adalah dengan menjelajahi setiap rute sedalam mungkin, saat mencapai tujuan, algoritma menambah jumlah jalur, lalu melakukan backtracking yaitu membatalkan status kunjungan pada sel tersebut agar rute alternatif lain dapat menggunakan sel yang sama. Karena pendekatan ini mencoba semua kombinasi langkah tanpa membatasi kunjungan sel, kompleksitas waktunya menjadi $2^{4R \times C}$ eksponensial, yakni $O(4^{R \times C})$, di mana setiap sel di dalam grid memiliki hingga empat potensi arah percabangan yang harus dievaluasi. Akibatnya, durasi eksekusi akan meningkat drastis seiring bertambahnya ukuran grid. Secara keseluruhan, karena semua struktur data yang digunakan matriks grid, matriks visited, dan stack skalanya berbanding lurus dengan total jumlah sel ($R \times C$), maka kompleksitas ruang totalnya adalah $O(R \times C)$.

Soal 4: DFS: Menghitung Semua Jalur Keluar Labirin

Time Limit: 2.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah labirin berukuran $R \times C$ dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma **Depth-First Search** (DFS) rekursif.

Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan.

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses **backtracking** agar dapat mencoba semua kemungkinan jalur yang valid.

Format Input

Input diberikan dalam format berikut.

```
R C
G11 G12 ... G1C G21 G22 ... G2C
...
GR1 GR2 ... GRC SR SC
FR FC
```

Keterangan:

R adalah jumlah baris grid.

C adalah jumlah kolom grid.

G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .

(S_R, S_C) adalah posisi awal.

(F_R, F_C) adalah posisi tujuan. Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

T

Keterangan:

T adalah jumlah jalur sederhana dari posisi awal ke posisi tujuan.

Jika tidak ada jalur yang dapat dilalui, cetak 0.

Constraints

$$1 \leq R \leq 7$$

$$1 \leq C \leq 7$$

G_{ij} hanya bernilai 0 atau 1.

$$0 \leq S_R < R$$

$$0 \leq S_C < C$$

$$0 \leq F_R < R$$

$$0 \leq F_C < C$$

Sel start dan finish dijamin bernilai 0.

Pergerakan hanya boleh dilakukan ke sel bernilai 0.

Satu jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.

Jumlah jalur valid pada test case dijamin tidak lebih dari 100000.

Algoritma yang digunakan wajib DFS rekursif dengan backtracking.

Contoh Kasus

Input	Output
5 6 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 1 0 0 0 0 0 0 4 5	4

Penjelasan Contoh

Posisi awal berada pada $(0,0)$ dan posisi tujuan berada pada $(4,5)$. Labirin memiliki lebih dari satu jalur valid serta beberapa cabang yang tidak menghasilkan solusi, misalnya cabang menuju sel $(0,5)$.

Dengan DFS dan backtracking, seluruh jalur sederhana dari start ke finish dapat dihitung. Pada contoh ini terdapat 4 jalur valid, sehingga output yang dicetak hanya 4.

A. Source Code

Table 4. Source Code Soal4.cpp

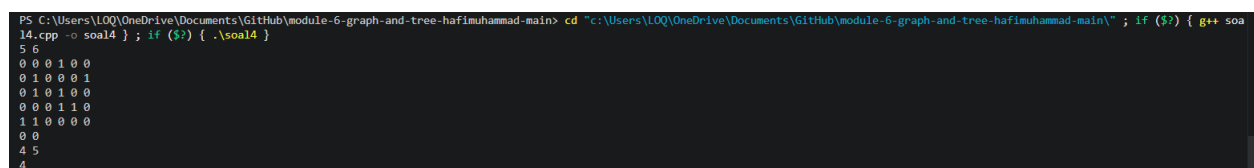
1.	#include <iostream>
2.	#include <vector>
3.	
4.	using namespace std;
5.	
6.	struct DataPosisi {
7.	int r, c;
8.	int arahKe;
9.	};
10.	
11.	int main() {
12.	int R, C;
13.	cin >> R >> C;
14.	
15.	vector<vector<int>> grid(R, vector<int>(C));
16.	for (int i = 0; i < R; ++i) {
17.	for (int j = 0; j < C; ++j) {
18.	cin >> grid[i][j];
19.	}
20.	}
21.	
22.	int SR, SC, FR, FC;
23.	cin >> SR >> SC >> FR >> FC;
24.	
25.	vector<vector<bool>> visited(R, vector<bool>(C,
	false));
26.	vector<DataPosisi> tumpukan;
27.	int jumlahJalur = 0;
28.	
29.	int dr[] = {-1, 1, 0, 0};
30.	int dc[] = {0, 0, -1, 1};
31.	
32.	tumpukan.push_back({SR, SC, 0});
33.	visited[SR][SC] = true;
34.	
35.	while (!tumpukan.empty()) {
36.	int r = tumpukan.back().r;
37.	int c = tumpukan.back().c;
38.	int d = tumpukan.back().arahKe;
39.	
40.	if (r == FR && c == FC) {
41.	jumlahJalur++;
42.	visited[r][c] = false;
43.	tumpukan.pop_back();
44.	continue;

```

45.         }
46.
47.         if (tumpukan.back().arahKe < 4) {
48.             int arahSekarang = tumpukan.back().arahKe;
49.             tumpukan.back().arahKe++;
50.
51.             int barisBaru = r + dr[arahSekarang];
52.             int kolomBaru = c + dc[arahSekarang];
53.
54.             if (barisBaru >= 0 && barisBaru < R &&
55.                 kolomBaru >= 0 && kolomBaru < C &&
56.                 grid[barisBaru][kolomBaru] == 0 &&
57.                 !visited[barisBaru][kolomBaru]) {
58.
59.                 visited[barisBaru][kolomBaru] = true;
60.                 tumpukan.push_back({barisBaru, kolomBaru,
0});
61.             }
62.         } else {
63.             visited[r][c] = false;
64.             tumpukan.pop_back();
65.         }
66.     }
67.
68.     cout << jumlahJalur << endl;
69.
70.     return 0;
71. }

```

B. Ouput Program



```

PS C:\Users\LOQ\OneDrive\Documents\Github\module-6-graph-and-tree-hafimuhammad-main> cd "c:\Users\LOQ\OneDrive\Documents\Github\module-6-graph-and-tree-hafimuhammad-main" ; if ($?) { g++ soa
l4.cpp -o soal4 } ; if ($?) { .\soal4 }
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4

```

Gambar 4. Ouput Soal4.cpp

C. Pembahasan

`#include <iostream>` dan `#include <vector>` ini dipakai untuk membuat pustaka standar C++ supaya program bisa melakukan input output dan menggunakan struktur data vector yang fleksibel.

`using namespace std;` ini bikin kita kda perlu nulis `std::` di depan setiap fungsi/objek standar library.

`struct DataPosisi { int r, c; int arahKe; };` di sini didefinisikan struct `DataPosisi` yang isinya tiga atribut: `r` dan `c` buat nyimpen posisi baris dan kolom suatu sel, sama `arahKe` buat nyimpen arah mana yang udah/akan dicoba dari sel itu (nilainya 0-4). Atribut `arahKe` ini yang bikin struct ini beda dari `Node` di kode sebelumnya — soalnya di sini kita perlu nginget "udah nyoba arah ke berapa" buat tiap sel yang ada di stack.

`int main() {` titik awal program mulai dijalankan.

`int R, C; cin >> R >> C;` deklarasi dan baca jumlah baris (`R`) dan kolom (`C`) dari grid. Beda dari kode BFS sebelumnya, di sini nggak ada validasi input (nggak dicek apakah pembacaan gagal atau `R/C`-nya negatif).

`vector<vector<int>> grid(R, vector<int>(C));` bikin grid dua dimensi ukuran `R x C` buat nyimpen data petanya.

`for (int i = 0; i < R; ++i) { for (int j = 0; j < C; ++j) { cin >> grid[i][j]; } }` dua loop bersarang buat baca isi grid dari input user, baris demi baris, kolom demi kolom.

`int SR, SC, FR, FC; cin >> SR >> SC >> FR >> FC;` deklarasi dan baca koordinat titik awal (`SR, SC`) dan titik tujuan (`FR, FC`). Sama seperti baris sebelumnya, di sini juga nggak ada validasi apakah koordinatnya masih dalam batas grid.

`vector<vector<bool>> visited(R, vector<bool>(C, false));` bikin matriks boolean ukuran `R x C`, semua diinisialisasi `false`, buat nyatat sel mana yang sedang dilewati di jalur yang lagi dijalanin sekarang.

`vector<DataPosisi> tumpukan;` bikin vector tumpukan yang berfungsi sebagai stack — di sinilah algoritmanya beda fundamental dari BFS, soalnya stack ini diakses dari ujung belakang (LIFO), bukan dari depan kayak antrean (FIFO).

`int jumlahJalur = 0;` variabel buat ngitung berapa banyak jalur yang berhasil nyampe dari titik awal ke titik tujuan.

`int dr[] = {-1, 1, 0, 0}; int dc[] = {0, 0, -1, 1};` dua array ini nyimpen pola pergeseran koordinat buat ngecek empat arah tetangga (atas, bawah, kiri, kanan), sama persis kayak di kode BFS.

`tumpukan.push_back({SR, SC, 0});` titik awal dimasukin ke stack, dengan `arahKe` diset 0 — artinya belum ada arah yang dicoba dari sel ini.

`visited[SR][SC] = true;` titik awal ditandai sebagai sedang dilewati.

`while (!tumpukan.empty()) {` loop utama jalan terus selama stack belum kosong.

`int r = tumpukan.back().r; int c = tumpukan.back().c; int d = tumpukan.back().arahKe;` ambil data posisi dan arah dari elemen paling atas (paling belakang) di stack — ini adalah sel yang sedang "aktif" diproses sekarang. Catatan: variabel `d` di sini sebenarnya nggak dipakai lagi di baris-baris berikutnya, soalnya nilai `arahKe` dibaca ulang langsung dari `tumpukan.back()` di kondisi `if` selanjutnya.

`if (r == FR && c == FC) {` kalau sel yang sedang aktif itu sama dengan titik tujuan, berarti satu jalur lengkap udah ketemu.

`jumlahJalur++;` jalur yang ketemu ini dihitung, nambahin total jumlah jalur.

`visited[r][c] = false; tumpukan.pop_back(); continue;` sel tujuan ini ditandai balik jadi `false` (di-unmark) dan dikeluarkan dari stack — ini bagian penting dari proses **backtracking**, soalnya sel ini harus "dilepas" lagi supaya bisa dilewati oleh jalur lain yang berbeda nantinya. Setelah itu `continue` lompat ke awal loop `while` buat lanjut proses elemen stack selanjutnya.

`if (tumpukan.back().arahKe < 4) {` kalau sel yang sekarang aktif masih punya arah yang belum dicoba (dari 4 arah yang mungkin), program lanjut nyoba arah berikutnya.

`int arahSekarang = tumpukan.back().arahKe;`
`tumpukan.back().arahKe++;` simpan arah yang mau dicoba sekarang ke `arahSekarang`, lalu langsung naikkan `arahKe` di elemen stack itu — jadi kalau nanti balik lagi ke sel ini, dia bakal coba arah selanjutnya, bukan ngulang arah yang sama.

`int barisBaru = r + dr[arahSekarang]; int kolomBaru = c + dc[arahSekarang];` ngitung koordinat tetangga ke arah yang sedang dicoba, berdasarkan posisi sekarang ditambah pola pergeseran dari array `dr` dan `dc`.

`if (barisBaru >= 0 && barisBaru < R && kolomBaru >= 0 && kolomBaru < C && grid[barisBaru][kolomBaru] == 0 && !visited[barisBaru][kolomBaru]) {` pengecekan empat syarat sebelum tetangga itu boleh dilewati: koordinatnya harus dalam batas grid, selnya harus jalan yang bisa dilewati (nilai grid 0), dan selnya belum sedang dilewati di jalur sekarang.

`visited[barisBaru][kolomBaru] = true;` kalau syaratnya terpenuhi, sel tetangga itu ditandai sebagai sedang dilewati.

`tumpukan.push_back({barisBaru, kolomBaru, 0});` sel tetangga itu dimasukin ke stack buat diproses, dengan `arahKe` diset 0 lagi karena belum ada arah yang dicoba dari sel baru ini.

`} }` dua kurung ini nutup blok `if` pengecekan syarat tetangga, lalu nutup blok `if` (`tumpukan.back().arahKe < 4`).

`} else { visited[r][c] = false; tumpukan.pop_back(); }` kalau ternyata sel yang sekarang aktif udah nyoba keempat arah (`arahKe` udah nyampe 4) dan nggak ada lagi yang bisa dieksplor, sel ini di-unmark dan dikeluarkan dari stack — ini juga bagian dari **backtracking**, mundur ke sel sebelumnya karena jalur dari sel ini udah buntu.

`}` kurung ini nutup loop `while` utama.

`cout << jumlahJalur << endl;` cetak total jumlah jalur yang ditemukan dari titik awal ke titik tujuan.

`return 0; }` fungsi main selesai.

D. Analisis

Algoritma yang dipakai adalah DFS dengan backtracking yang berfungsi menghitung semua jumlah dari titik awal ke tujuan. Berbeda dari BFS yang akan berhenti ketika tujuannya itu ketemu pertama kali, kalo DFS ini programnya itu harus tetap melanjutkan eksplorasi sampai semua kemungkinan rute habis dicoba. Karena tujuannya bukan jalur terpendek tapi total jumlah jalur yang sudah ada

Cara kerjanya setiap sel yang masuk ke stack di siman bersama atribut `arahKe` yang menyimpan progres arah mana yang sudah di coba dari sel itu. Selama sebuah sel masih mempunyai arah yang belum dicoba (`arahKe < 4`), program bakal terus nyoba arah berikutnya satu per satu. Begitu sel itu nyampe ke titik tujuan, jumlah Jalur ditambah, lalu sel itu **di-unmark** (visited diset balik ke false) dan dikeluarkan dari stack — proses unmark inilah yang disebut backtracking, soalnya sel itu harus "dilepas" supaya bisa dipakai lagi oleh rute lain yang berbeda. Backtracking yang sama juga terjadi kalau sebuah sel udah nyoba keempat arah dan nggak ada lagi yang bisa dieksplor (jalur buntu).

Untuk kompleksitas ruang, karena semua struktur data utama yang dipakai — grid, matriks visited, dan stack tumpukan — skalanya berbanding lurus dengan total jumlah sel ($R \times C$), maka kompleksitas ruang totalnya adalah $O(R \times C)$.

Soal 5: Tree Traversal

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan N bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah Red-Black Tree secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree.

Setelah RBTree terbentuk, diberikan Q query. Setiap query meminta salah satu hasil traversal berikut.

PREORDER, yaitu urutan root, subtree kiri, lalu subtree kanan.

INORDER, yaitu urutan subtree kiri, root, lalu subtree kanan.

POSTORDER, yaitu urutan subtree kiri, subtree kanan, lalu root.

ALL, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

Format Input

Input diberikan dalam format berikut.

```
N
A1 A2 ... AN Q
T1 T2
... TQ
```

Keterangan:

N adalah banyaknya bilangan yang akan dimasukkan ke RBTree.

A_i adalah bilangan ke- i .

Q adalah banyaknya query traversal.

T_i adalah jenis traversal yang diminta.

Format Output

Untuk setiap query, cetak hasil traversal sesuai format berikut.

```
[Preorder] : daftar_angka [Inorder] : daftar_angka [Postorder] : daftar_angka
```

Jika query adalah ALL, cetak ketiga baris traversal dengan urutan Preorder, Inorder, lalu Postorder.

Jika setelah penghapusan duplikat tree tetap kosong, cetak:

Tree kosong. Tidak ada yang bisa ditampilkan.

Constraints

$$0 \leq N \leq 1000$$

$$-10000000000 \leq A_i \leq 10000000000$$

$$1 \leq Q \leq 20$$

T_i hanya salah satu dari PREORDER, INORDER, POSTORDER, atau ALL.

Contoh Kasus

Input	Output
7	[Preorder] : 50 30 20 40 70 60 80
50 30 70 20 40 60 80	[Inorder] : 20 30 40 50 60 70 80
4	[Postorder] : 20 40 30 60 80 70 50
PREORDER INORDER POSTORDER ALL	[Preorder] : 50 30 20 40 70 60 80
	[Inorder] : 20 30 40 50 60 70 80
	[Postorder] : 20 40 30 60 80 70 50

Penjelasan Contoh

Bilangan dimasukkan ke RBTree secara berurutan. Nilai 50 menjadi root. Nilai yang lebih kecil dari 50 masuk ke subtree kiri, sedangkan nilai yang lebih besar masuk ke subtree kanan.

Hasil INORDER pada RBTree selalu menghasilkan urutan nilai dari kecil ke besar. Pada contoh ini, hasil inorder adalah 20 30 40 50 60 70 80, sehingga struktur RBTree yang dibentuk sudah konsisten dengan aturan RBTree.

A. Source Code

Table 5. Source Code Soal5.cpp

1.	#include <iostream>
2.	#include <string>
3.	#include "RedBlackTree.h"
4.	
5.	using namespace std;
6.	
7.	void cetakPreorder(const RedBlackTree::Node* node, bool &first) {
8.	if (node->isNil) return;
9.	
10.	if (!first) cout << " ";
11.	cout << node->key;
12.	first = false;
13.	
14.	cetakPreorder(node->left, first);
15.	cetakPreorder(node->right, first);
16.	}
17.	
18.	void cetakInorder(const RedBlackTree::Node* node, bool &first) {
19.	if (node->isNil) return;
20.	
21.	cetakInorder(node->left, first);
22.	
23.	if (!first) cout << " ";
24.	cout << node->key;
25.	first = false;
26.	
27.	cetakInorder(node->right, first);
28.	}
29.	
30.	void cetakPostorder(const RedBlackTree::Node* node, bool &first) {
31.	if (node->isNil) return;
32.	
33.	cetakPostorder(node->left, first);
34.	cetakPostorder(node->right, first);
35.	
36.	if (!first) cout << " ";
37.	cout << node->key;
38.	first = false;
39.	}
40.	
41.	int main() {
42.	RedBlackTree rbt;
43.	int N;
44.	if (!(cin >> N)) return 0;
45.	


```

46.     for (int i = 0; i < N; i++) {
47.         int angka;
48.         cin >> angka;
49.         if (!rbt.contains(angka)) {
50.             rbt.insert(angka);
51.         }
52.     }
53.
54.     int Q;
55.     cin >> Q;
56.     for (int i = 0; i < Q; i++) {
57.         string tipeQuery;
58.         cin >> tipeQuery;
59.
60.         bool first = true;
61.         if (tipeQuery == "PREORDER") {
62.             cout << "[Preorder] : ";
63.             cetakPreorder(rbt.root(), first);
64.             cout << endl;
65.         } else if (tipeQuery == "INORDER") {
66.             cout << "[Inorder] : ";
67.             cetakInorder(rbt.root(), first);
68.             cout << endl;
69.         } else if (tipeQuery == "POSTORDER") {
70.             cout << "[Postorder] : ";
71.             cetakPostorder(rbt.root(), first);
72.             cout << endl;
73.         } else if (tipeQuery == "ALL") {
74.             first = true;
75.             cout << "[Preorder] : ";
76.             cetakPreorder(rbt.root(), first);
77.             cout << endl;
78.
79.             first = true;
80.             cout << "[Inorder] : ";
81.             cetakInorder(rbt.root(), first);
82.             cout << endl;
83.
84.             first = true;
85.             cout << "[Postorder] : ";
86.             cetakPostorder(rbt.root(), first);
87.             cout << endl;
88.         }
89.     }
90.
91.     return 0;
92. }

```

B. Output Program

```
PS C:\Users\LOQ\OneDrive\Documents\Github\module-6-graph-and-tree-hafimuhammad-main> g++ -std=c++11 soal5.cpp RedBlackTree.cpp -o soal5.exe
PS C:\Users\LOQ\OneDrive\Documents\Github\module-6-graph-and-tree-hafimuhammad-main> ./soal5.exe
7
50 30 70 20 40 60 80
4
PREORDER INORDER POSTORDER ALL
[Preorder] : 50 30 20 40 70 60 80
[Inorder]   : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50
[Preorder] : 50 30 20 40 70 60 80
[Inorder]   : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50
```

Gambar 5. Output Soal5.cpp

C. Pembahasan

`#include <iostream>`, `#include <string>`, dan `#include "RedBlackTree.h"` dipakai buat memasukkan pustaka standar C++ untuk input output dan tipe data string, sekaligus header custom `RedBlackTree.h` yang isinya implementasi struktur data Red-Black Tree-nya sendiri (kelas `RedBlackTree` dengan fungsi-fungsi kayak `insert`, `contains`, dan `root`).

`using namespace std;` supaya tidak perlu nulis `std::` di depan tiap fungsi/objek standar library.

`void cetakPreorder(const RedBlackTree::Node* node, bool &first) {` deklarasi fungsi rekursif buat nyetak isi tree dengan urutan preorder (root dulu, baru kiri, baru kanan). Parameternya berupa pointer ke node yang sedang diproses, dan referensi ke boolean `first` yang dipakai buat ngatur spasi antar angka pas dicetak.

`if (node->isNil) return;` ini base case dari rekursi — kalau node yang sekarang itu node "nil" (semacam node penanda kosong yang biasa dipakai di implementasi Red-Black Tree, gantinya `nullptr`), fungsi langsung berhenti tanpa nyetak apa-apa.

`if (!first) cout << " "; cout << node->key; first = false;` kalau ini bukan angka pertama yang dicetak, kasih spasi dulu sebelum nyetak key dari node sekarang, lalu tandain `first` jadi false biar angka selanjutnya juga dikasih spasi pemisah.

`cetakPreorder(node->left, first); cetakPreorder(node->right, first);` dua pemanggilan rekursif ke anak kiri dan anak kanan — sesuai urutan preorder, root udah dicetak duluan di atas, baru sekarang lanjut ke subtree kiri, lalu subtree kanan.

`void cetakInorder(const RedBlackTree::Node* node, bool &first) {` fungsi rekursif buat nyetak isi tree dengan urutan inorder (kiri dulu, baru root, baru kanan) — strukturnya mirip `cetakPreorder`, tapi posisi pencetakan key-nya beda.

`if (node->isNil) return;` base case yang sama, berhenti kalau ketemu node nil.

`cetakInorder(node->left, first);` rekursi ke subtree kiri duluan, sebelum nyetak node sekarang — ini yang bikin hasilnya keluar dalam urutan teratur (khususnya buat BST/RBT yang teratur berdasarkan key).

if (!first) cout << " "; cout << node->key; first = false; setelah subtree kiri selesai diproses semua, baru key dari node sekarang dicetak.

cetakInorder(node->right, first); terakhir, rekursi ke subtree kanan.

void cetakPostorder(const RedBlackTree::Node* node, bool &first) { fungsi rekursif buat nyetak isi tree dengan urutan postorder (kiri dulu, kanan, baru root paling akhir).

if (node->isNil) return; base case yang sama.

cetakPostorder(node->left, first); cetakPostorder(node->right, first); rekursi ke subtree kiri dan kanan duluan, sebelum nyetak node sekarang.

if (!first) cout << " "; cout << node->key; first = false; key dari node sekarang baru dicetak paling akhir, setelah kedua subtree-nya selesai diproses semua.

int main() { titik awal program mulai dijalankan.

RedBlackTree rbt; bikin objek rbt dari kelas RedBlackTree yang struktur internalnya didefinisikan di file header di sinilah treenya bakal dibangun.

int N; if (!(cin >> N)) return 0; baca jumlah angka yang mau dimasukin ke tree, langsung berhenti kalau pembacaannya gagal.

for (int i = 0; i < N; i++) { perulangan buat masukan N angka satu per satu ke dalam tree.

int angka; cin >> angka; baca satu angka dari input.

if (!rbt.contains(angka)) { rbt.insert(angka); } sebelum dimasukin, dicek dulu apakah angka itu udah ada di tree (pakai fungsi contains) — kalau belum ada, baru di-insert. Ini buat mencegah duplikat key di dalam tree.

int Q; if (!(cin >> Q)) return 0; baca jumlah query yang mau diproses, langsung berhenti kalau pembacaannya gagal.

for (int i = 0; i < Q; i++) { perulangan buat memproses Q query satu per satu.

string tipeQuery; cin >> tipeQuery; baca jenis query yang diminta, dalam bentuk string (misalnya "PREORDER", "INORDER", dst).

bool first = true; variabel first direset ke true di tiap awal query baru, supaya format spasi pas pencetakan tree mulai dari kosong lagi setiap kali ada query baru.

```

if (tipeQuery == "PREORDER") { cout << "[Preorder] : ";
cetakPreorder(rbt.root(), first); cout << endl; } kalau query-nya "PREORDER",
cetak label dulu, lalu panggil fungsi cetakPreorder dari root tree, terus pindah baris baru.

} else if (tipeQuery == "INORDER") { cout << "[Inorder] : ";
cetakInorder(rbt.root(), first); cout << endl; } mirip di atas, tapi buat query
"INORDER".

} else if (tipeQuery == "POSTORDER") { cout << "[Postorder] : ";
cetakPostorder(rbt.root(), first); cout << endl; } mirip juga, buat query
"POSTORDER".

} else if (tipeQuery == "ALL") { kalau query-nya "ALL", program bakal nyetak ketiga
jenis traversal sekaligus dalam satu kali query.

first = true; cout << "[Preorder] : "; cetakPreorder(rbt.root(), first); cout
<< endl; cetak preorder dulu — first di-reset lagi ke true di sini supaya format spasinya
bener buat traversal yang baru ini, soalnya kalau nggak di-reset, status first bisa dibawa dari
traversal sebelumnya.

first = true; cout << "[Inorder] : "; cetakInorder(rbt.root(), first); cout
<< endl; lanjut cetak inorder, dengan first di-reset lagi.

first = true; cout << "[Postorder] : "; cetakPostorder(rbt.root(), first);
cout << endl; terakhir cetak postorder, juga dengan first direset lagi.

} else { cout << "Query tidak dikenali: " << tipeQuery << endl; } kalau tipe
query yang dibaca nggak cocok sama salah satu dari empat pilihan di atas, program nyetak pesan
error yang nunjukin query apa yang nggak dikenali.

} kurung ini nutup loop for pemrosesan query.

return 0; } menandakan fungsi main selesai dan program berhasil dijalankan sampai akhir.

} else if (tipeQuery == "ALL") { kalau query-nya "ALL", program bakal nyetak ketiga jenis
traversal sekaligus dalam satu kali query.

first = true; cout << "[Preorder] : "; cetakPreorder(rbt.root(), first); cout << endl; cetak preorder
dulu — first di-reset lagi ke true di sini supaya format spasinya benar buat traversal yang baru ini,
soalnya kalau nggak di-reset, status first bisa dibawa dari traversal sebelumnya.

first = true; cout << "[Inorder] : "; cetakInorder(rbt.root(), first); cout << endl; lanjut cetak inorder,
dengan first di-reset lagi.

first = true; cout << "[Postorder] : "; cetakPostorder(rbt.root(), first); cout << endl; terakhir cetak
postorder, juga dengan first direset lagi.

```

```
} else { cout << "Query tidak dikenali: " << tipeQuery << endl; } kalau tipe query yang dibaca  
nggak cocok sama salah satu dari empat pilihan di atas, program nyetak pesan error yang nunjukin  
query apa yang nggak dikenali.
```

```
} kurung ini nutup loop for pemrosesan query.
```

```
return 0; } fungsi main selesai.
```

D. Analisis

cara yang dipakai adalah tree traversal pada Red-Black Tree dengan menggunakan rekursi untuk menelusuri setiap node, di mana setiap metode (Preorder, Inorder, dan Postorder) memiliki urutan kunjungan yang berbeda berdasarkan prioritas kunjungan antara akar, cabang kiri, dan cabang kanan. Kompleksitas waktunya adalah $O(N)$ karena algoritma wajib mengunjungi seluruh node tree sebanyak satu kali untuk keperluan print di layar, sementara kompleksitas ruangnya adalah $O(\log N)$ karena sifat Red-Black Tree yang selalu menjaga keseimbangan tinggi pohon melalui rotasi otomatis. Pada fungsi main, program mengelola alur dengan membaca N data yang disaring melalui metode contains, kemudian merespons Q perintah query secara dinamis, di mana variabel `bool &first` diterapkan sebagai pengendali pemformatan spasi agar setiap output tetap rapi dan tidak menyertakan spasi di awal atau akhir baris.